

МДК.01.04 Системное программирование

Модули

модуль random

Случайные числа

Случайные числа – последовательность чисел, в которой невозможно предсказать следующее число, зная все предыдущие.

Случайные числа широко используются в различных задачах программирования:

- в играх (имитация подбрасывания игрального кубика и другие подобные ситуации);
- в программах имитационного моделирования;
- в статистических программах, случайным образом отбирающих данные для анализа;
- в компьютерной безопасности для шифрования уязвимых данных.

Для создания истинно случайных чисел можно бросать монету, игральные кости или измерять какой-нибудь шумовой сигнал.

Псевдослучайные числа

Ставить сложные электронные приборы на каждый компьютер для генерации истинно случайных чисел дорого, поэтому математики и программисты создали алгоритмы получения **псевдослучайных чисел**.

Для обычного человека псевдослучайные числа практически не отличаются от случайных, однако они вычисляются по математической формуле, и зная первое число и формулу, можно определить любое следующее.

Python предлагает встроенные функции для работы с псевдослучайными числами. Эти функции хранятся в модуле `random` в стандартной библиотеке.

Модуль random

Модуль `random` предоставляет функции для генерации псевдослучайных чисел, букв и случайного выбора элементов последовательности (списка, строки и так далее).

Для использования этих функций в начале программы необходимо подключить модуль, что делается командой `import` :

```
import random
```

После подключения модуля мы можем использовать его функции.

Функция randint()

Функция `randint()` принимает два обязательных аргумента `a` и `b` и возвращает псевдослучайное целое число из отрезка $[a; b]$.

Приведенный ниже код выводит два псевдослучайных целых числа: `num1` из отрезка $[0; 17]$ и `num2` из отрезка $[-5; 5]$.

```
import random

num1 = random.randint(0, 17)
num2 = random.randint(-5, 5)
```



Левая и правая граница `a` и `b` включаются в диапазон генерируемых псевдослучайных чисел. Результатом вызова функции `random.randint(2, 9)` может быть любое число от 2 до 9, включая 2 и 9.

Приведенный ниже код выводит 10 псевдослучайных целых чисел из диапазона $[1; 100]$:

```
import random

for _ in range(10):
    print(random.randint(1, 100))
```

Среди этих чисел возможны повторения.

Функция `randrange()`

Если вы помните, как применять функцию `range()`, то почувствуете себя непринужденно с функцией `randrange()`. Функция `randrange()` принимает такие же аргументы, что и функция `range()`. Различие состоит в том, что функция `randrange()` не возвращает саму последовательность чисел. Вместо этого она возвращает случайно выбранное число из последовательности чисел.

Приведенный ниже код присваивает переменной `num` псевдослучайное число в диапазоне от 0 до 9:

```
import random

num = random.randrange(10)
```

Аргумент 10 задает конечный предел последовательности значений. Функция возвратит случайно выбранное число из последовательности чисел от 0 до конечного предела, исключая сам предел.

Приведенный ниже код задает начальное значение и конечный предел последовательности:

```
import random

num = random.randrange(5, 10)
```

Таким образом, в переменной `num` будет храниться псевдослучайное число в диапазоне от 5 до 9.

Приведенный ниже код задает начальное значение, конечный предел и величину шага:

```
import random

num = random.randrange(0, 101, 10)
```

Таким образом, в переменной `num` будет храниться псевдослучайное число из последовательности чисел: 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100.

Функция random()

Функции `randint()` и `randrange()` возвращают псевдослучайное **целое число**. А вот функция `random()` возвращает псевдослучайное число с плавающей точкой (вещественное число). В функцию `random()` никаких аргументов не передается. Функция `random()` возвращает случайное число с плавающей точкой в диапазоне от 0.0 до 1.0 (исключая 1.0).

Приведенный ниже код выводит случайное число с плавающей точкой из диапазона [0.0; 1.0):

```
import random

num = random.random()
print(num)
```

Функция uniform()

Функция `uniform()` тоже возвращает случайное число с плавающей точкой, но при этом она позволяет задавать диапазон для отбора значений.

Следующий код выводит псевдослучайное число с плавающей точкой из диапазона [1.5; 17.3] (включительно):

```
import random

num = random.uniform(1.5, 17.3)
print(num)
```

Функция seed()

Как уже было сказано **псевдослучайные числа** вычисляются на основе некой формулы. Генерация случайных чисел иницируется **начальным значением**. Оно используется в вычислении, возвращающем следующее случайное число в ряду. Когда модуль `random` импортируется, он получает системное время из внутреннего генератора тактовых импульсов компьютера и использует его как начальное значение. Системное время – целое число, представляющее собой текущую дату и время вплоть до сотой секунды. Если бы всегда использовалось одно и то же начальное значение, функции генерации случайных чисел всегда возвращали бы один и тот же ряд псевдослучайных чисел. Поскольку системное время меняется каждую сотую долю секунды, можно утверждать, что всякий раз, когда импортируется модуль `random`, будет создана отличающаяся от предыдущих последовательность случайных чисел.

Вместе с тем, некоторые программы требуют генерации одной и той же последовательности случайных чисел. Для этого можно вызвать функцию `seed()`, задав начальное значение.

Следующий код генерирует 10 псевдослучайных чисел, и при этом содержит инструкцию, явно устанавливающую начальное значение для генератора случайных чисел:

```
import random

random.seed(17)    # явно устанавливаем начальное значение для генератора
                   # случайных чисел

for _ in range(10):
    print(random.randint(1, 100))
```

Приведенный выше код выводит:

```
67
54
39
47
38
23
99
91
91
70
```

Если выполнить код еще раз, получим ту же самую последовательность псевдослучайных чисел.

Метод `shuffle()`

Метод `shuffle()` принимает список в качестве обязательного аргумента и перемешивает его случайным образом.

Следующий код перемешивает список `numbers` случайным образом, а затем выводит его содержимое:

```
import random

numbers = [1, 2, 3, 4, 5, 6, 7, 8]
random.shuffle(numbers)
print(numbers)
```

Результатом работы такого кода может быть:

```
[4, 7, 8, 1, 2, 3, 6, 5]
```

Метод choice()

Метод `choice()` принимает список (строку, кортеж) в качестве обязательного аргумента и возвращает **один** случайный элемент.

Следующий код выводит по одному случайному элементу из строки `'BEEGEEK'`, списка `[1, 2, 3, 4]` и кортежа `('a', 'b', 'c', 'd')`:

```
import random

print(random.choice('BEEGEEK'))
print(random.choice([1, 2, 3, 4]))
print(random.choice(('a', 'b', 'c', 'd')))
```

Результатом работы такого кода может быть:

```
E
3
c
```

Метод sample()

Метод `sample()` принимает два обязательных аргумента: первый – коллекция (последовательность), которая поддерживает индексацию (список, строка, кортеж), второй – количество случайных элементов. Возвращает **список** из указанного количества уникальных (имеющих разные индексы) случайных элементов.

Результатом работы кода:

```
import random

numbers = [2, 5, 8, 9, 12]

print(random.sample(numbers, 1))
print(random.sample(numbers, 2))
print(random.sample(numbers, 3))
print(random.sample(numbers, 5))
```

может быть:

```
[9]
[12, 5]
[9, 2, 8]
[12, 8, 9, 5, 2]
```

Количество случайных элементов не должно превышать длину исходного списка (строки). Следующий код:

```
import random

numbers = [2, 5, 8, 9, 12]

print(random.sample(numbers, 6))
```

приведет к ошибке:

```
ValueError: Sample larger than population or is negative
```

Модуль string

Встроенный модуль `string` раньше использовался для расширения стандартных возможностей (функционала) строкового типа данных `str`. На текущий момент все функции из модуля `string` переехали в методы строкового типа данных `str`, однако в модуле `string` остались удобные константные строки, которые можно использовать при решении задач.

Приведенный ниже код:

```
import string

print(string.ascii_letters)
print(string.ascii_uppercase)
print(string.ascii_lowercase)
print(string.digits)
print(string.hexdigits)
print(string.octdigits)
print(string.punctuation)
print(string.printable)
```

ВЫВОДИТ:

```
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
0123456789
0123456789abcdefABCDEF
01234567
!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~ \t\n\r\x0b\x0c
```


модуль decimal

Числовые типы данных

В прошлых уроках мы изучили два числовых типа данных, представленных в Python:

- `int` – целое число;
- `float` – число с плавающей точкой (аналог вещественного числа в математике).

В Python есть три дополнительных числовых типа данных:

- `Decimal` – десятичное число, для выполнения точных расчетов;
- `Fraction` – число, представляющее собой [обыкновенную дробь](#), с заданным числителем и знаменателем;
- `Complex` – комплексное число.

В этом уроке мы изучим числовой тип данных `Decimal`, аналог типа данных `float` на случай более точных вычислений.

Тип данных float

Рассмотрим программный код:

```
if 0.3 == 0.3:
    print('YES')
else:
    print('NO')
```

Результатом выполнения такого кода будет, как и полагается, `YES`.

А теперь рассмотрим программный код:

```
num = 0.1 + 0.1 + 0.1

if num == 0.3:
    print('YES')
else:
    print('NO')
```

Вы будете удивлены, но результатом выполнения такого кода будет `NO`, так как на самом деле в переменной `num` хранится что-то типа `0.30000000000000004`.

Из-за ограничений в сохранении точного значения чисел даже простейшие математические операции могут выдавать ошибочный результат. Поэтому, чтобы сравнивать два `float` числа, мы должны использовать такой код:

```
num = 0.1 + 0.1 + 0.1
eps = 0.000000001          # точность сравнения

if abs(num - 0.3) < eps:    # число num отличается от числа 0.3 менее
    print('YES')            чем 0.000000001
else:
    print('NO')
```

Такой код выводит, как полагается, значение `YES`.

Не стоит сравнивать `float` числа с помощью оператора `==`. Для сравнения `float` чисел нужно использовать приведенный выше код.

Тип данных Decimal

Тип данных `Decimal` – это класс из стандартного модуля `decimal`. Он представляет собой число с плавающей точкой, как и `float`. Однако `Decimal` имеет ряд существенных отличий от `float`.

Тип `Decimal` создан, чтобы операции над вещественными числами в компьютере выполнялись как в математике, и равенство $0.1 + 0.1 + 0.1 == 0.3$ было верным.

Точность результатов арифметических действий очень важна для научных вычислений, в сфере финансов и бизнеса. Для таких задач тип данных `float` не подходит.

В Python тип данных `float` реализован по стандарту IEEE-754 как число с плавающей точкой двойной точности (64 бита) с основанием экспоненты равным 2. Реализация таких чисел заложена прямо в железо любого современного процессора. Поэтому `float` в Python работает как аналогичный тип данных `double` в таких языках программирования, как C#, C++, Java и так далее. И имеет такие же ограничения и «странности». Так как `float` поддерживается аппаратно, быстродействие при использовании этого типа данных сравнительно велико.

Тип данных `Decimal` – число с плавающей точкой с основанием экспоненты 10. Он реализован по стандарту IBM: General Decimal Arithmetic Specification, в свою очередь основанному на стандартах IEEE.

Тип данных `Decimal` реализован программно, поэтому он в разы медленнее типа данных `float`, реализованного аппаратно. Сам тип данных `Decimal` написан на языке C.

Тип данных `Decimal` оперирует числами с произвольной – задаваемой программистом, но конечной точностью. По умолчанию точность составляет 28 десятичных знаков.

Тип данных `Decimal` – неизменяемый. Операции над ним приводят к созданию новых объектов, при этом старые не меняются.

Еще одно следствие того, что `Decimal` реализован программно – его можно на ходу настраивать как угодно программисту. Для этого есть контекст – объект, содержащий настройки для выполнения операций. Операции, выполняемые в контексте, следуют заданным в нем правилам. Для `float` все правила фиксированы на аппаратном уровне.

Для типа данных `Decimal` можно настроить:

- точность выполнения операций в количестве десятичных знаков;
- режимы округления;
- режимы обработки исключительных ситуаций (деление на ноль, переполнение и так далее).

Создание `Decimal` чисел

Создать `Decimal` число можно из обычного целого числа (`int`), из числа с плавающей точкой (`float`) или из строки (`str`).

Приведенный ниже программный код создает `Decimal` числа на основе целого числа и строки:

```
from decimal import *

d1 = Decimal(1)
d2 = Decimal(567)
d3 = Decimal(-93)
d4 = Decimal('12345')
d5 = Decimal('52.198')

print(d1, d2, d3, d4, d5, sep='\n')
```

и выводит:

```
1
567
-93
12345
52.198
```

При создании `Decimal` чисел из чисел с плавающей точкой (`float`) возникают проблемы, так как `float` числа округляются внутри до ближайшего возможного, а `Decimal` об этом ничего не знает и копирует содержимое `float` .

Приведенный ниже код:

```
from decimal import *  
  
num = Decimal('5.2')  
  
print(num + 1)  
print(num - 10)  
print(num * 2)  
print(num ** 4)
```

ВЫВОДИТ:

```
6.2  
-4.8  
10.4  
731.1616
```

Математические функции

`Decimal` числа можно передавать как аргументы функциям, ожидающим `float`. Они будут преобразованы во `float`. К примеру, модуль `math`, оперирующий `float` числами, может работать и с `Decimal` числами.

Приведенный ниже код:

```
from decimal import *  
from math import *  
  
num1 = Decimal('1.44')  
num2 = Decimal('0.523')  
  
print(sqrt(num1))  
print(sin(num2))  
print(log(num1 + num2))
```

ВЫВОДИТ:

```
1.2  
0.4994813555186418  
0.6744739152943241
```

Важно понимать, что результатом работы функции модуля `math` являются `float` числа, а не `Decimal`.

Тип данных `Decimal` содержит некоторые встроенные математические методы, возвращающие значения `Decimal`.

Сравнение float и Decimal чисел

Выбор между типами данных `Decimal` и `float` – поиск компромисса в условиях конкретной задачи.

Если нужно считать очень много (симуляции, физика, графика, игры), имеет смысл отказаться от точности `Decimal` в пользу скорости и компактности хранения данных `float`. В бизнесе и финансах считать приходится не так много, но делать это нужно предельно точно, тут имеет смысл посмотреть в сторону `Decimal`.

Характеристика / тип	float	Decimal
Реализация	аппаратная	программная
Размер	64 бит	не ограничен
Основание экспоненты	2	10
Скорость	✓	✗
Настраиваемость	✗	✓
Для финансов и бизнеса	✗	✓
Для симуляций, визуализаций и игр	✓	✗
Для высокоточных вычислений	✗	✓